Dimensionality Reduction

In machine learning, "dimensionality" simply refers to the number of features (i.e. input variables) in the dataset.

When the number of features is very large relative to the number of observations in your dataset, certain algorithms struggle to train effective models. This is called the "**Curse of Dimensionality**", and it's especially relevant for clustering algorithms that rely on distance calculations.

We have 2 primary methods for reducing dimensionality:

1. Feature Selection
2. Feature Extraction.

Feature Selection: - Feature selection is for filtering irrelevant or redundant features from your dataset. The key difference between feature selection and extraction is that feature selection keeps a subset of the original features while feature extraction creates brand new ones.

To be clear, some supervised algorithms already have **built-in feature selection**, such as Regularized Regression and Random Forests. As a stand-alone task, feature selection can be unsupervised (e.g. Variance Thresholds) or supervised (e.g. Genetic Algorithms). You can also combine multiple methods if needed.

- **Variance Thresholds**

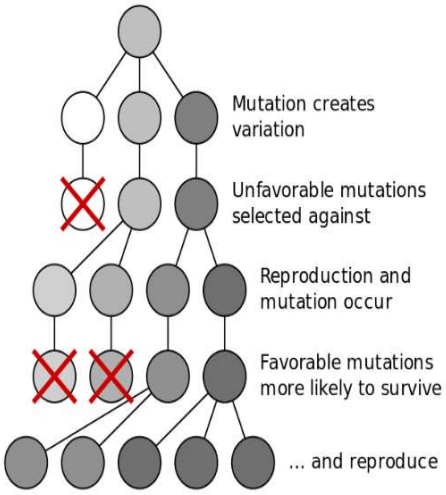
  Variance thresholds remove features whose values don't change much from observation to observation (i.e. their variance falls below a threshold). These features provide little value. Because variance is dependent on scale, you should always normalize your features first.

- **Correlation Thresholds**

  Correlation thresholds remove features that are highly correlated with others (i.e. its values change very similarly to another's). These features provide redundant information.

- **Genetic Algorithm**

  Genetic algorithms (GA) are a broad class of algorithms that can be adapted to different purposes. They are *search algorithms* that are inspired by evolutionary biology and natural selection, combining mutation and cross-over to efficiently traverse large solution spaces. Here's a great intro to the intuition behind GA's.



In machine learning, GA's have two main uses. The first is for *optimization*, such as finding the best weights for a neural network.

The second is for supervised feature selection. In this use case, "genes" represent individual features and the "organism" represents a candidate set of features. Each organism in the "population" is graded on a fitness score such as model performance on a hold-out set. The fittest organisms survive and reproduce, repeating until the population converges on a solution some generations later.

- **Honorable Mention: Stepwise Search**

  Stepwise search is a supervised feature selection method based on sequential search, and it has two flavours: forward and backward.

  For forward stepwise search, you start without any features. Then, you'd train a 1-feature model using each of your candidate features and keep the version with the best performance. You'd continue adding features, one at a time, until your performance improvements stall.

  Backward stepwise search is the same process, just reversed: start with all features in your model and then remove one at a time until performance starts to drop substantially.

Feature Extraction: - Feature extraction is for creating a new, smaller set of features that stills captures most of the useful information. Again, feature selection keeps a subset of the original features while feature extraction creates new ones.

As with feature selection, some algorithms already have **built-in feature extraction**. The best example is Deep Learning, which extracts increasingly useful representations of the raw input data through each hidden neural layer.

As a stand-alone task, feature extraction can be unsupervised (i.e. PCA) or supervised (i.e. LDA).

- **Principal Component Analysis (PCA)**

  Principal component analysis (PCA) is an unsupervised algorithm that creates linear combinations of the original features. The new features are orthogonal, which means that they are uncorrelated. Furthermore, they are ranked in order of their "explained variance." The *first principal component* (PC1) explains the most variance in your dataset, PC2 explains the second-most variance, and so on.

  Therefore, you can reduce dimensionality by limiting the number of principal components to keep based on cumulative explained variance.

  You should always normalize your dataset before performing PCA because the transformation is dependent on scale. If you don't, the features that are on the largest scale would dominate your new principal components.

- **Linear Discriminant Analysis (LDA)**

  Linear discriminant analysis (LDA) - not to be confused with latent Dirichlet allocation - also creates linear combinations of your original features. However, unlike PCA, LDA doesn't maximize explained variance. Instead, it maximizes the *separability* between classes.

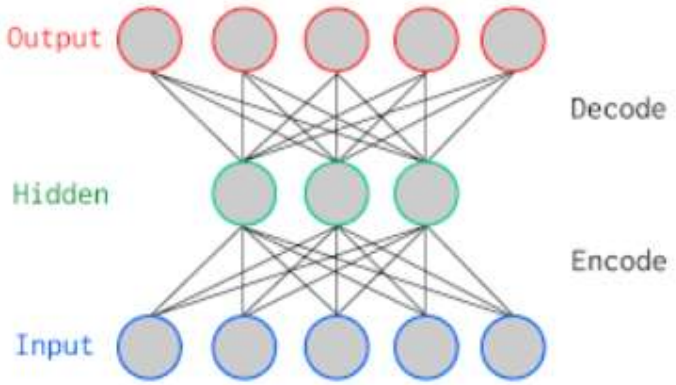
  Therefore, LDA is a supervised method that can only be used with labeled data. So which is better: LDA and PCA? Well, results will vary from problem to problem, and the same "No Free Lunch" theorem.

  The LDA transformation is also dependent on scale, so you should normalize your dataset first.

- **Autoencoders**

  Autoencoders are neural networks that are trained to reconstruct their original inputs. For example, image autoencoders are trained to reproduce the original images instead of classifying the image as a dog or a cat.

  So how is this helpful? Well, the key is to structure the hidden layer to have *fewer neurons* than the input/output layers. Thus, that hidden layer will learn to produce a smaller representation of the original image.



Because you use the input image as the target output, autoencoders are considered unsupervised. They can be used directly (e.g. image compression) or stacked in sequence (e.g. deep learning).

No Free Lunch Theorem: -

In machine learning, there's something called the "No Free Lunch" theorem. In a nutshell, it states that no one algorithm works best for every problem, and it's especially relevant for supervised learning (i.e. predictive modelling).

For example, you can't say that neural networks are always better than decision trees or vice-versa. There are many factors at play, such as the size and structure of your dataset.

As a result, you should **try many different algorithms for your problem**, while using a hold-out "test set" of data to evaluate performance and select the winner.

Of course, the algorithms you try must be appropriate for your problem, which is where picking the right machine learning task comes in. As an analogy, if you need to clean your house, you might use a vacuum, a broom, or a mop, but you wouldn't bust out a shovel and start digging.